

```
1  from nodo import Nodo
2
3
4  class Lista_encadeada:
5      """
6      Lista simplesmente encadeada.
7      Implementa métodos de inserção, remoção, busca e os métodos exigidos na prova.
8      """
9
10     def __init__(self):
11         self.cabeca = None
12
13     def vazia(self):
14         return self.cabeca is None
15
16     def inserir_inicio(self, info):
17         novo = Nodo(info)
18         novo.prox = self.cabeca
19         self.cabeca = novo
20
21     def inserir_final(self, info):
22         novo = Nodo(info)
23
24         if self.vazia():
25             self.cabeca = novo
26         else:
27             aux = self.cabeca
28
29             while aux.prox:
30                 aux = aux.prox
31
32             aux.prox = novo
33
34     def imprimir(self):
35         if self.vazia():
36             print("Lista vazia")
37             return
38
39         aux = self.cabeca
40
41         while aux:
42             print(aux.info)
43             aux = aux.prox
44
45     def buscar(self, valor):
46         aux = self.cabeca
47
48         while aux:
49             if aux.info == valor:
50                 return True
51
52             aux = aux.prox
53
```

```

54         return False
55
56     def remover_inicio(self):
57         if not self.vazia():
58             aux = self.cabeca
59             self.cabeca = self.cabeca.prox
60             del aux
61
62     def remover_final(self):
63         if self.vazia():
64             return
65
66         aux = self.cabeca
67         ant = None
68
69         while aux.prox:
70             ant = aux
71             aux = aux.prox
72
73         if ant is None:
74             self.cabeca = None
75         else:
76             ant.prox = None
77
78         del aux
79
80     def remover_elemento(self, valor):
81         if self.vazia():
82             print("Lista vazia")
83             return
84
85         aux = self.cabeca
86         ant = None
87
88         while aux and aux.info != valor:
89             ant = aux
90             aux = aux.prox
91
92         if aux is None:
93             print("Elemento não encontrado")
94             return
95
96         if ant is None:
97             self.cabeca = aux.prox
98         else:
99             ant.prox = aux.prox
100
101         del aux
102         print("Elemento removido")
103
104     def esvaziar(self):
105         while not self.vazia():
106             self.remover_inicio()
107
108     # -----

```

```
109 # MÉTODOS EXIGIDOS NA PROVA - LISTA
110 # -----
111
112 def min(self):
113     """
114     Retorna o menor elemento da lista.
115     """
116     if self.vazia():
117         return None
118
119     menor = self.cabeca.info
120     aux = self.cabeca.prox
121
122     while aux:
123         if aux.info < menor:
124             menor = aux.info
125
126         aux = aux.prox
127
128     return menor
129
130 def max(self):
131     """
132     Retorna o maior elemento da lista.
133     """
134     if self.vazia():
135         return None
136
137     maior = self.cabeca.info
138     aux = self.cabeca.prox
139
140     while aux:
141         if aux.info > maior:
142             maior = aux.info
143
144         aux = aux.prox
145
146     return maior
147
148 def sum(self):
149     """
150     Retorna a soma dos elementos numéricos da lista.
151     """
152     total = 0
153     aux = self.cabeca
154
155     while aux:
156         total += aux.info
157         aux = aux.prox
158
159     return total
160
161 def avg(self):
162     """
163     Retorna a média dos elementos numéricos da lista.
```

```
164         """
165         quantidade = self.len()
166
167         if quantidade == 0:
168             return None
169
170         return self.sum() / quantidade
171
172     def len(self):
173         """
174         Retorna a quantidade de elementos da lista.
175         """
176         contador = 0
177         aux = self.cabeca
178
179         while aux:
180             contador += 1
181             aux = aux.prox
182
183         return contador
184
185     def append(self, outra_lista):
186         """
187         Concatena outra lista ao final da lista principal.
188         Os elementos são copiados para evitar dependência entre as listas.
189         """
190         aux = outra_lista.cabeca
191
192         while aux:
193             self.inserir_final(aux.info)
194             aux = aux.prox
195
196     def reverse(self):
197         """
198         Inverte a ordem dos elementos da lista.
199         """
200         anterior = None
201         atual = self.cabeca
202
203         while atual:
204             proximo = atual.prox
205             atual.prox = anterior
206             anterior = atual
207             atual = proximo
208
209         self.cabeca = anterior
210
```